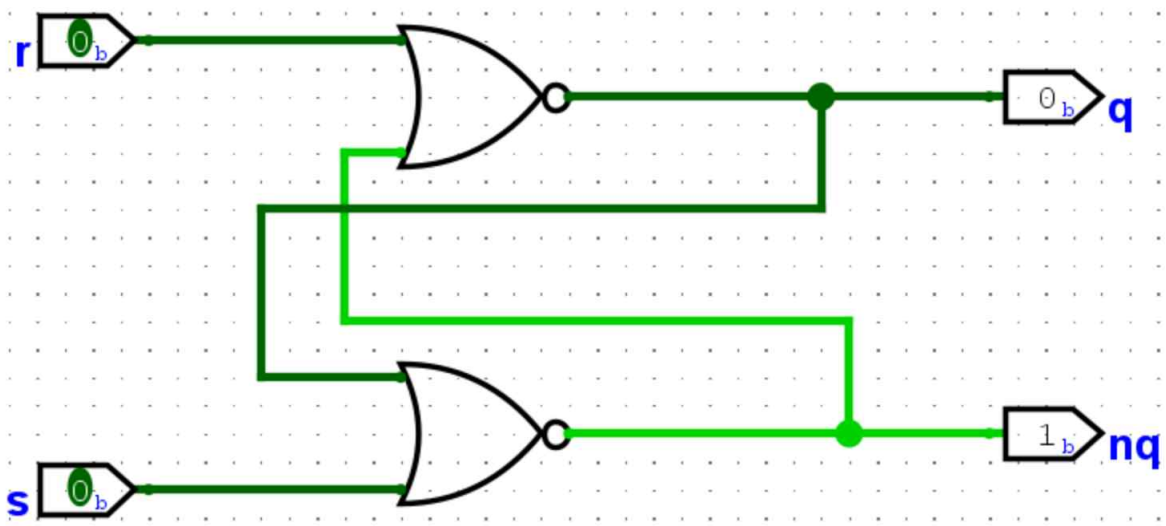
디지털논리회로설계

과제_3

2023047992 천지용

1. RS Latch

a. gate level implementation



r과 s를 입력받아 NOR 게이트를 통과하게 만든 RS latch 회로이다. 서로의 출력을 입력으로 받는 피드백 구조로 값을 저장한다.

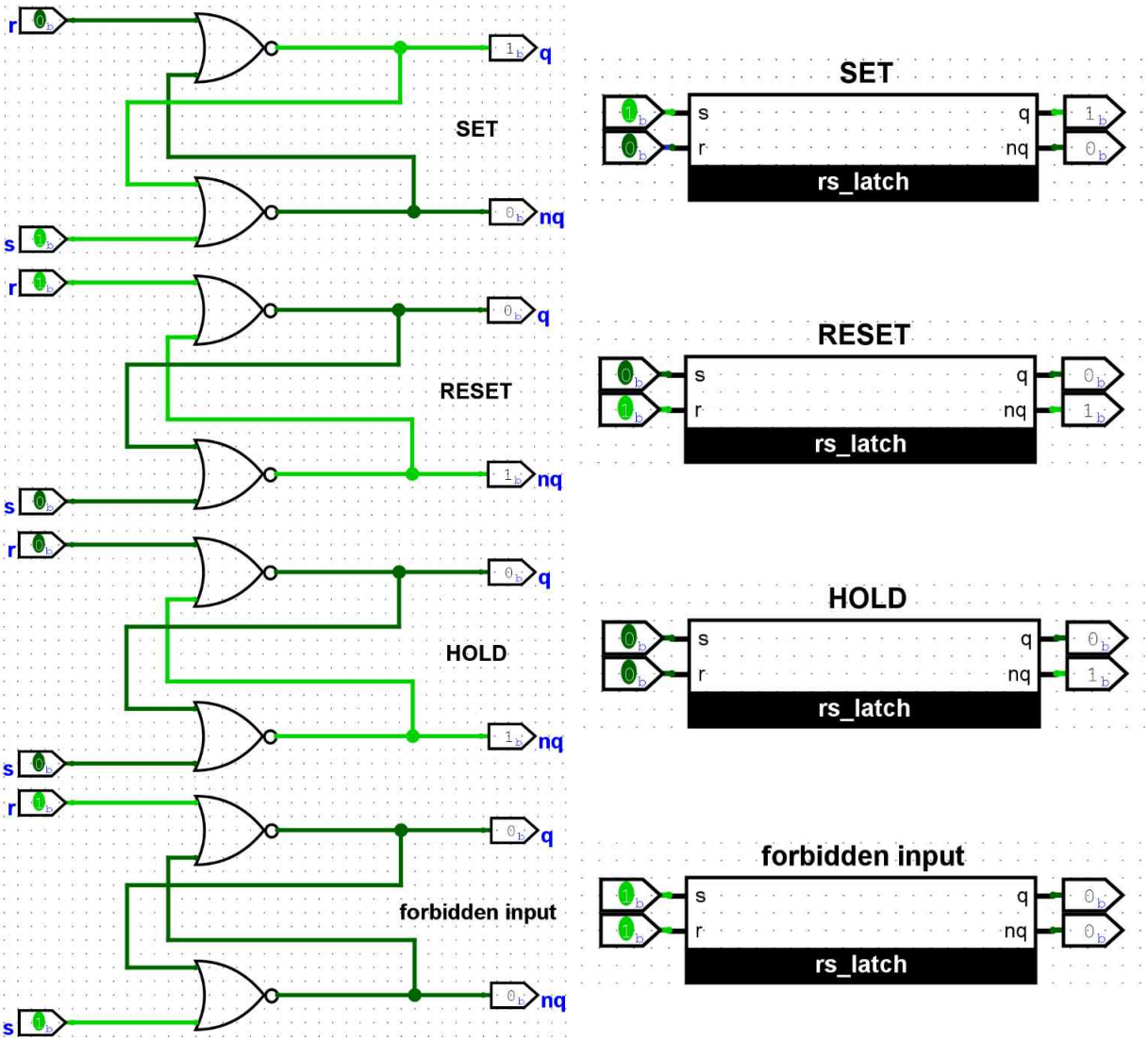
b. VHDL based implementation

assignment_3_1.vhd

```
1   LIBRARY ieee;
2   USE ieee.std_logic_1164.ALL;
3
4   ENTITY rs_latch IS
5   PORT (
6       s  : IN std_logic;
7       r  : IN std_logic;
8       q  : OUT std_logic;
9       nq : OUT std_logic
10  );
11  END rs_latch;
12
13  ARCHITECTURE bhv OF rs_latch IS
14  BEGIN
15
16      PROCESS(s, r)
17      BEGIN
18          IF (s = '1' AND r = '0') THEN
19              q  <= '1';
20              nq <= '0';
21          ELSIF (s = '0' AND r = '1') THEN
22              q  <= '0';
23              nq <= '1';
24          ELSIF (s = '1' AND r = '1') THEN
25              q  <= '0';
26              nq <= '0';
27          END IF;
28      END PROCESS;
29
30  END bhv;
31
```

c. simulation screenshots

gate level (Logisim) **VHDL (Logisim VHDL entity)**



set: r에 0, s에 1을 입력하여 q에 1이 출력되는 것을 확인할 수 있다.

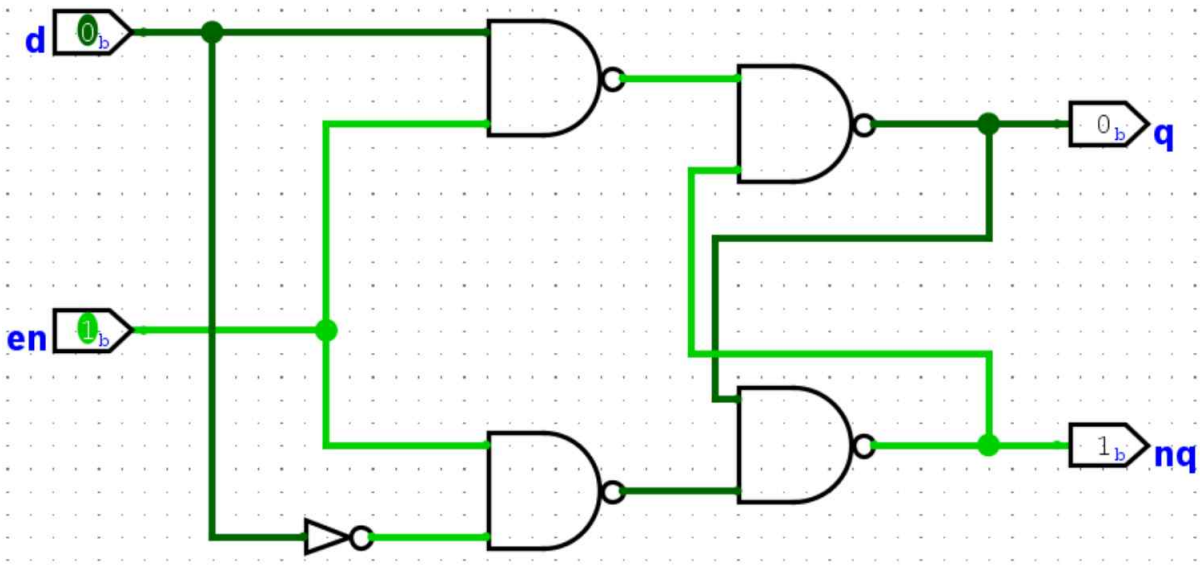
reset: r에 1, s에 0을 입력하여 nq에 1이 출력되고, q에 0이 출력되는 것을 확인할 수 있다.

hold: r의 입력이 0으로 바뀌었지만, nq에 출력된 1은 저장되어 남아있는 것을 확인할 수 있다.

forbidden input: r, s에 1이 입력되어 저장되었던 값이 지워지고 q, nq가 0을 출력하는 것을 확인할 수 있다.

2. D Latch

a. gate level implementation



d와 en을 입력으로 받아 NAND 게이트 4개와 인버터로 D latch를 구성했다. 앞단의 NAND 두 개는 en이 1일 때만 d와 d'를 뒷단으로 전달하고, 뒷단의 NAND 두 개는 서로의 출력을 입력으로 받는 피드백 구조로 값을 저장한다.

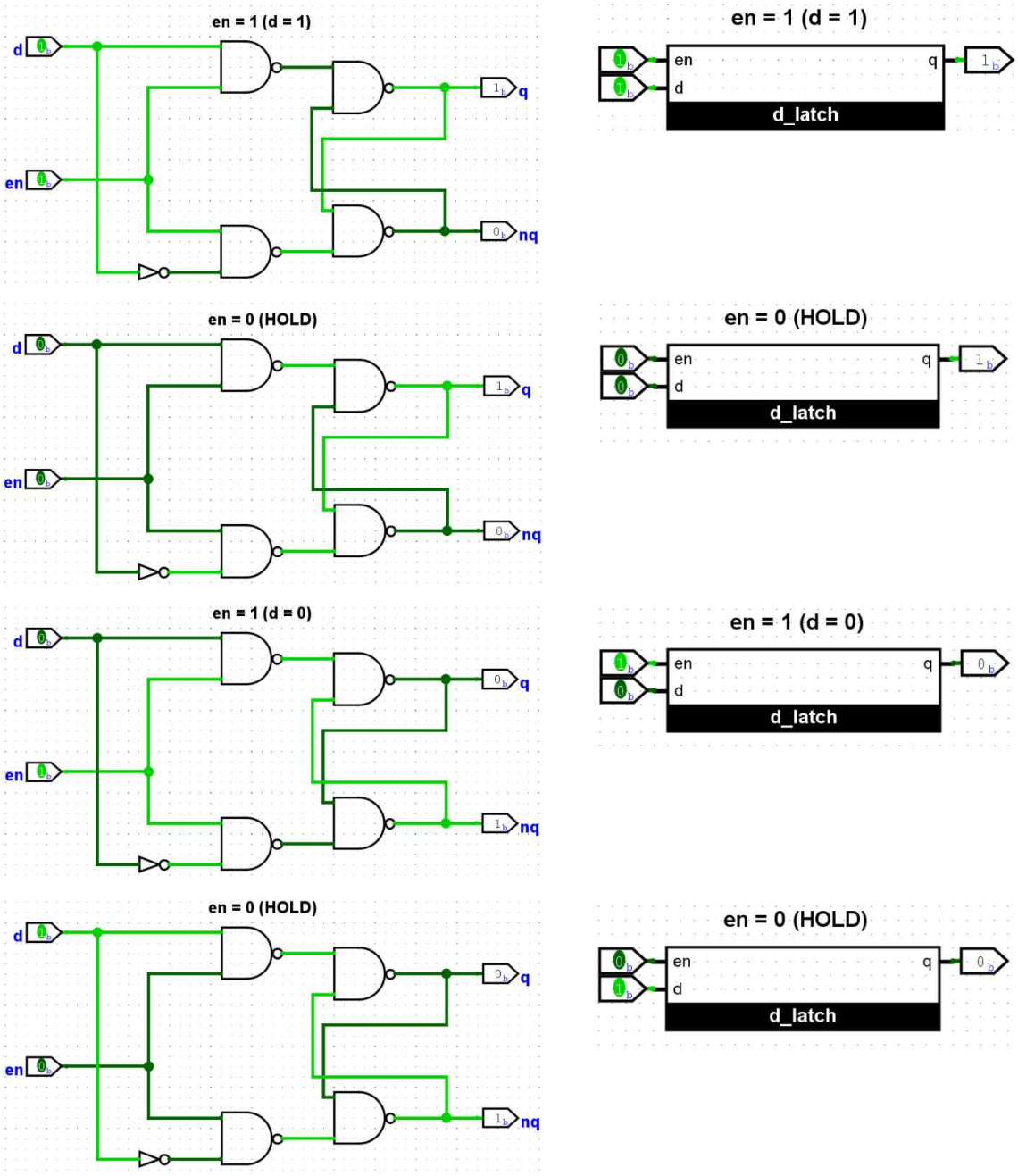
b. VHDL based implementation

assignment_3_2.vhd

```
1   LIBRARY ieee;
2   USE ieee.std_logic_1164.ALL;
3
4   ENTITY d_latch IS
5   PORT (
6       en : IN std_logic;
7       d  : IN std_logic;
8       q  : OUT std_logic
9   );
10  END d_latch;
11
12  ARCHITECTURE bhv OF d_latch IS
13  BEGIN
14
15      PROCESS(en, d)
16      BEGIN
17          IF (en = '1') THEN
18              q <= d;
19          END IF;
20      END PROCESS;
21
22  END bhv;
23
```

c. simulation screenshots

gate level (Logisim) | **VHDL (Logisim VHDL entity)**



en = 1, d = 1: en에 1, d에 1을 입력하여 q에 1이 출력되는 것을 확인할 수 있다.

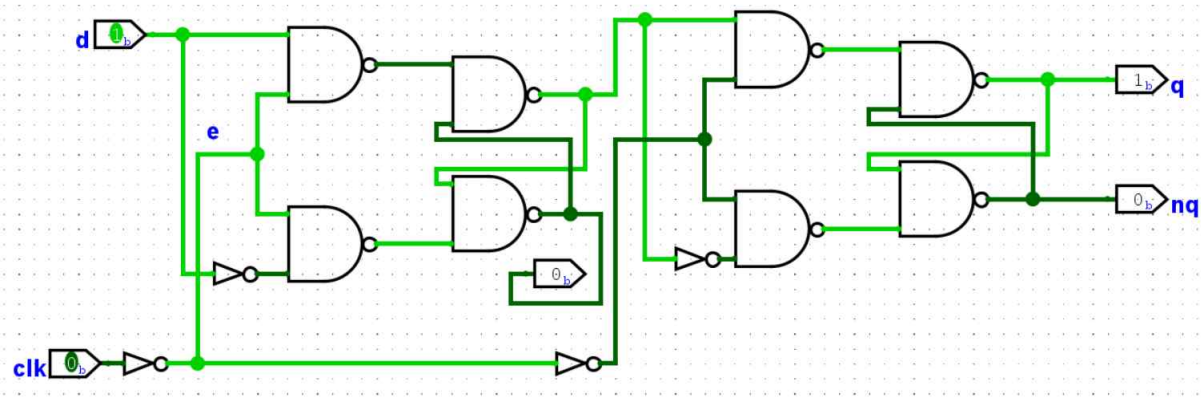
en = 0 (HOLD): en을 0으로 바꾼 뒤 d를 0으로 바꾸었지만, q는 이전 값인 1을 그대로 유지하는 것을 확인할 수 있다.

en = 1 (ㅇ = 0): en에 다시 1을 입력하자 q가 d를 따라 0으로 바뀌는 것을 확인할 수 있다.

en = 0 (HOLD): en이 0인 상태에서는 d에 1을 입력해도 q가 0을 유지하는 것을 확인할 수 있다.

3. D Flip-Flop

a. gate level implementation



2번의 D latch 두 개를 직렬로 연결하여 Master-Slave D Flip-Flop을 구성했다. clk를 NOT으로 반전한 신호(e)가 마스터 latch의 enable로 들어가고, 이 신호를 한 번 더 반전한 clk가 슬레이브 latch의 enable로 들어간다. 따라서 clk=0일 때는 마스터가 d를 받아들이고, clk가 1로 올라가는 순간 마스터에 저장된 값이 슬레이브로 넘어가 q에 출력된다. 가운데의 출력 핀은 마스터 latch의 nq 값을 확인하기 위해 연결했다.

b. VHDL based implementation

assignment_3_3.vhd

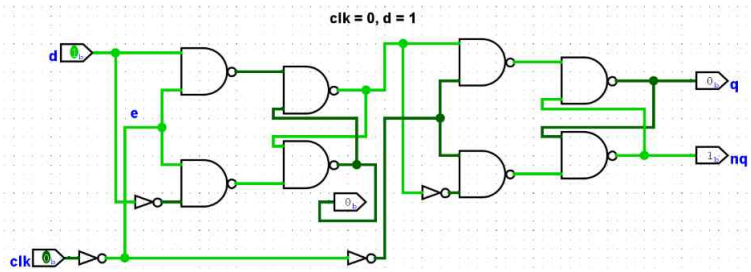
```
1   LIBRARY ieee;
2   USE ieee.std_logic_1164.ALL;
3
4   ENTITY d_flipflop IS
5     PORT (
6       clk : IN std_logic;
7       d   : IN std_logic;
8       q   : OUT std_logic
9     );
10  END d_flipflop;
11
12
13  ARCHITECTURE bhv OF d_flipflop IS
14  BEGIN
15    process(clk)
16    BEGIN
17      IF (clk'event AND clk = '1') THEN
18        q <= d;
19      END IF;
20    END process;
21  END bhv;
22
```

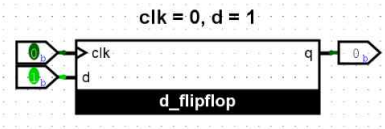
process의 감지 목록에 clk만 넣고, clk가 0에서 1로 바뀌는 상승 에지에서만 q <= d가 수행되도록 했다.

c. simulation screenshots

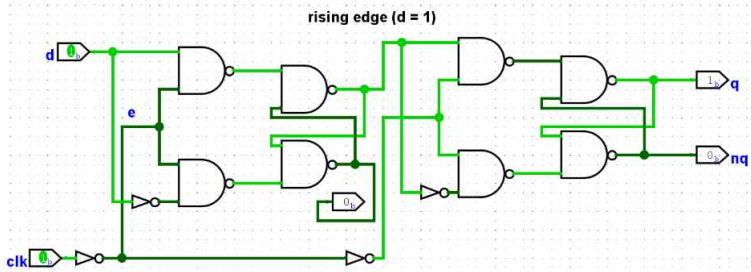
gate level (Logisim) | **VHDL (Logisim VHDL entity)**

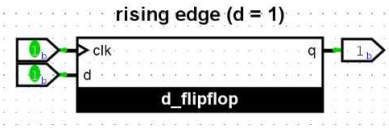
clk = 0, d = 1



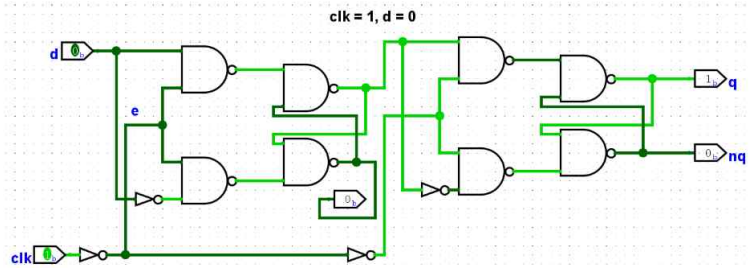


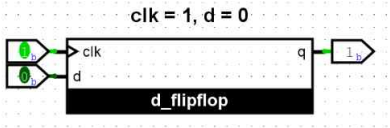
rising edge (d = 1)



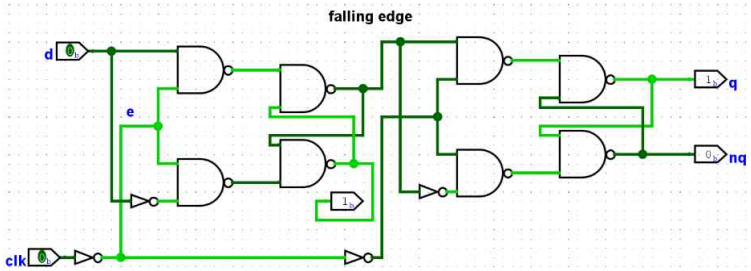


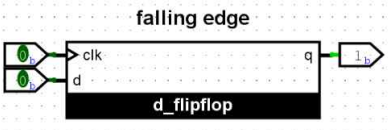
clk = 1, d = 0



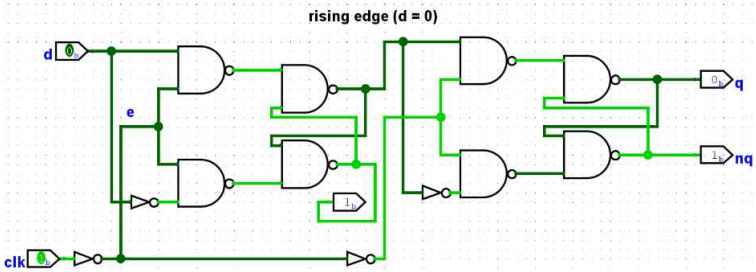


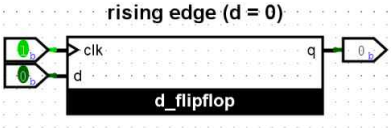
falling edge





rising edge (d = 0)





clk = 0, d = 1: clk가 0인 동안 d에 1을 입력했다. 마스터 latch는 d를 받아들이지만 슬레이브 latch는 닫혀 있어 q는 0을 유지하는 것을 확인할 수 있다.

rising edge (d = 1): clk가 0에서 1로 바뀌는 순간 d의 값을 받아들여 q가 1로 바뀌는 것을 확인할 수 있다.

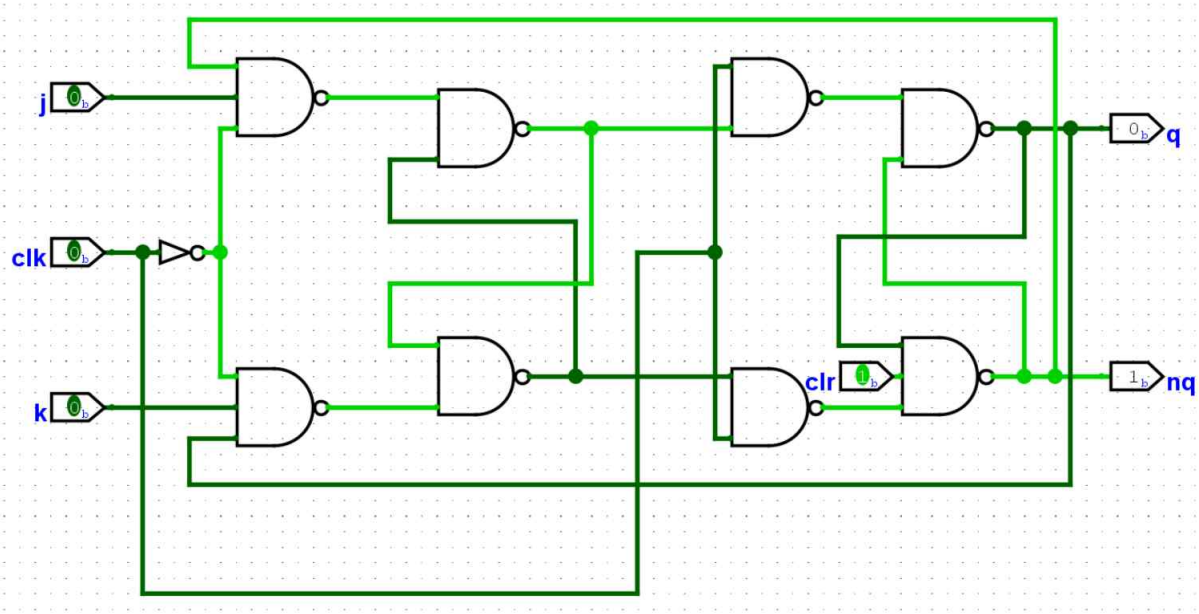
clk = 1, d = 0: clk가 1인 상태에서 d를 0으로 바꾸었지만 q는 1을 유지한다. 상승 에지가 아니면 d의 변화가 출력에 반영되지 않는 것을 확인할 수 있다.

falling edge: clk가 1에서 0으로 떨어져도 q는 바뀌지 않는 것을 확인할 수 있다.

rising edge (d = 0): 다음 상승 에지에서 d의 값 0이 q에 반영되는 것을 확인할 수 있다.

4. JK Flip-Flop

a. gate level implementation



3입력 NAND 2개와 NAND 2개로 Master latch를, NAND 4개로 Slave latch를 구성한 Master-Slave JK Flip-Flop이다. 출력 q와 nq를 입력단의 3입력 NAND로 되먹임(feedback)하여 j=k=1일 때 출력이 반전되도록 했다. 슬레이브는 clk = 0일 때, 마스터는 NOT으로 반전된 clk를 받아 clk = 1일 때 동작한다. clr은 회로를 처음 실행했을 때 출력이 정해지지 않는 것(데드락)을 막기 위한 초기화용 입력으로, 0을 입력하면 nq를 1로 만들어 q를 0으로 초기화하고 평상시에는 1로 둔다.

b. VHDL based implementation

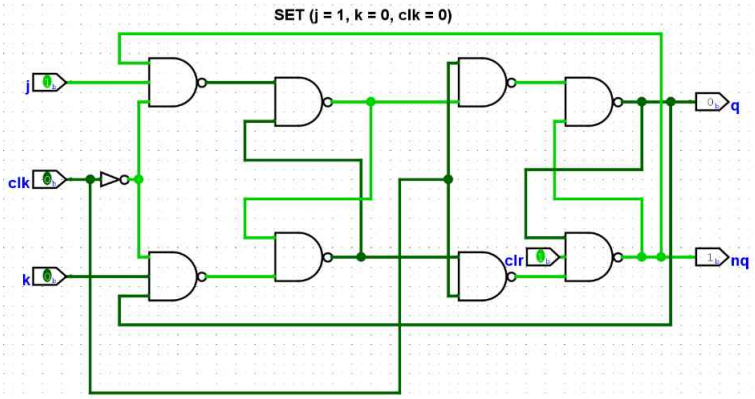
assignment_3_4.vhd

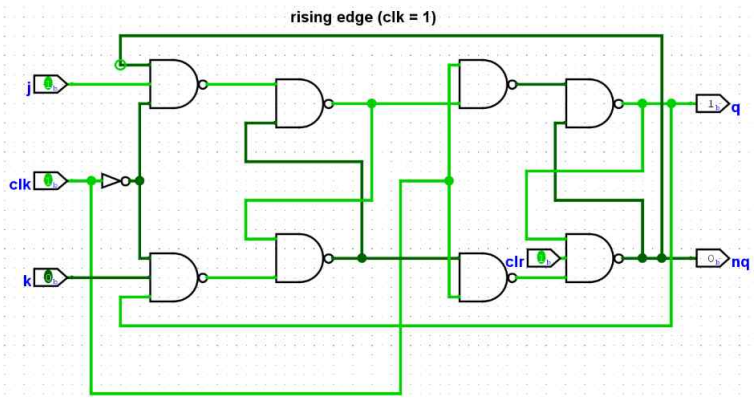
```
14  END ENTITY jk_flipflop;
15
16  ARCHITECTURE bhv OF jk_flipflop IS
17    SIGNAL q_internal : std_logic := '0';
18  BEGIN
19    q <= q_internal;
20    process(clk, clr, set)
21    begin
22      if clr = '1' then
23        q_internal <= '0';
24      elsif set = '1' then
25        q_internal <= '1';
26      elsif (clk'event and clk = '1') then
27        if (j = '1' and k = '0') then
28          q_internal <= '1';
29        elsif (j = '0' and k = '1') then
30          q_internal <= '0';
31        elsif (j = '1' and k = '1') then
32          q_internal <= not q_internal;
33        end if;
34      end if;
35    end process;
36  END ARCHITECTURE bhv;
37
```

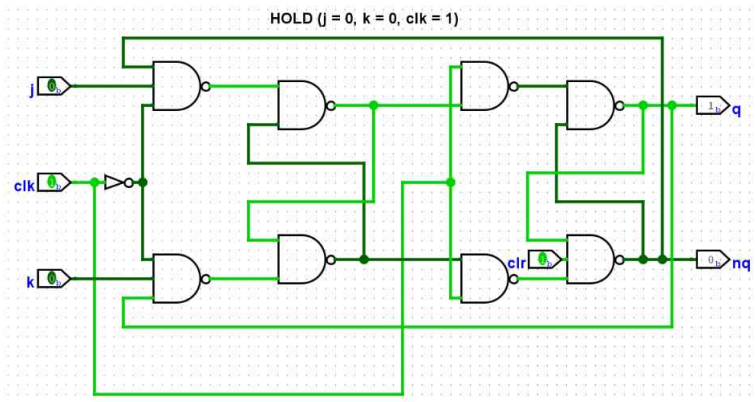
j, k 값에 따라 if문으로 네 가지 동작을 구분했다. (j, k)가 (1, 0)이면 '1', (0, 1)이면 '0', (1, 1)이면 not q_internal로 반전하고, (0, 0)이면 아무 값도 대입하지 않아 이전 값을 유지한다. out 포트인 q는 다시 읽을 수 없기 때문에 내부 signal q_internal에 값을 저장하고 q로 내보냈다. 추가로 비동기 clr, set 입력을 두어 clk와 관계없이 출력을 0 또는 1로 만들 수 있게 했으며, 아래 j/k 동작 확인에서는 clr, set을 모두 0으로 두었다.

c. simulation screenshots

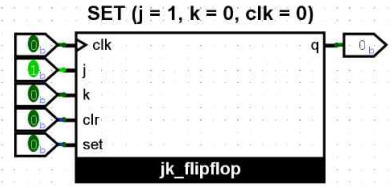
gate level (Logisim)

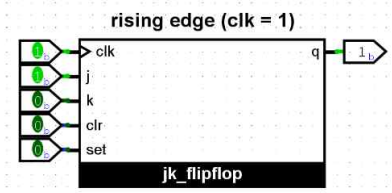


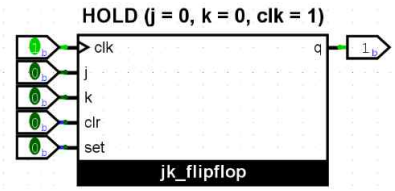




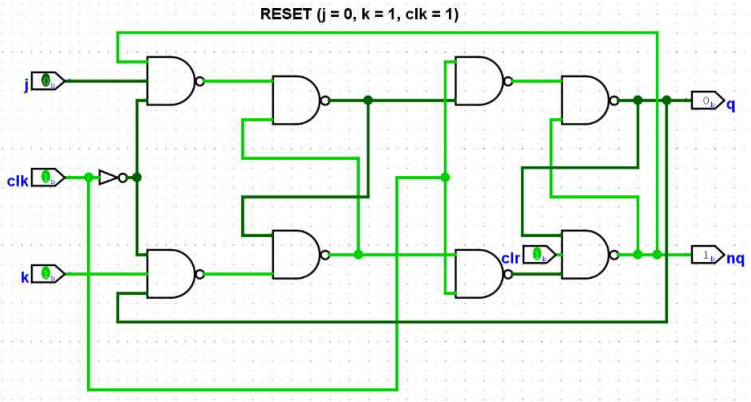
VHDL (Logisim VHDL entity)







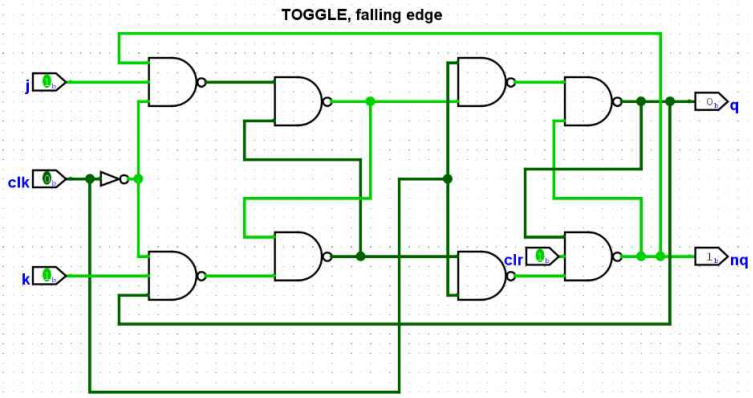
RESET (j = 0, k = 1, clk = 1)



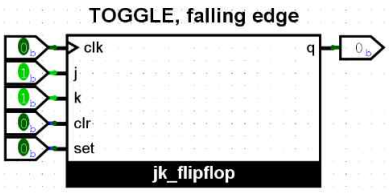
RESET (j = 0, k = 1, clk = 1)



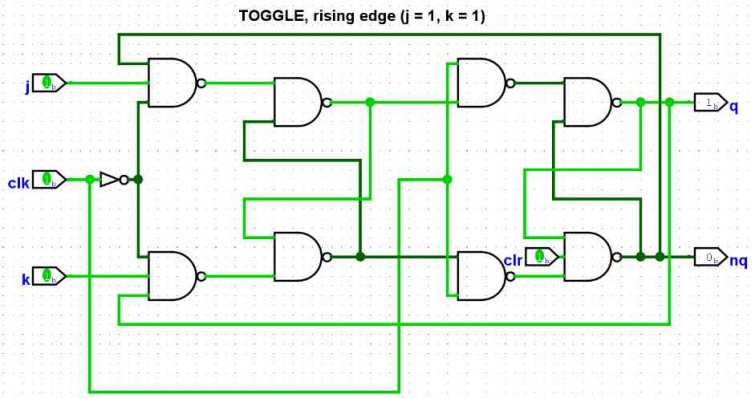
TOGGLE, falling edge



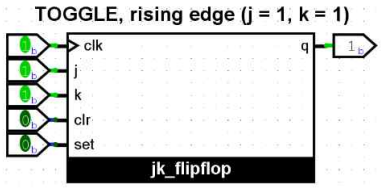
TOGGLE, falling edge



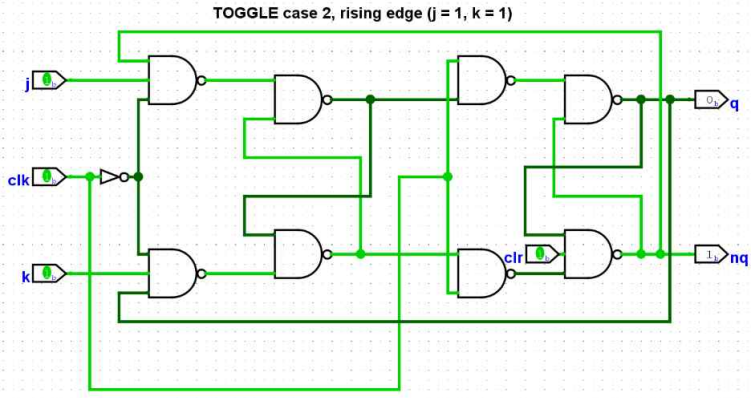
TOGGLE, rising edge (j = 1, k = 1)



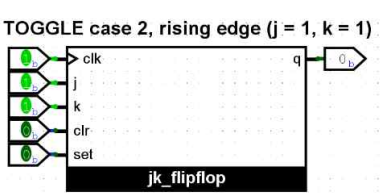
TOGGLE, rising edge (j = 1, k = 1)



TOGGLE case 2, rising edge (j = 1, k = 1)



TOGGLE case 2, rising edge (j = 1, k = 1)



SET: j = 1, k = 0을 입력하고 clk 입력을 하여 q에 1이 출력되는 것을 확인할 수 있다. 이때 상승 엣지로 입력됨을 확인할 수 있다.

HOLD: j = 0, k = 0일 때는 clk 입력이 들어와도 q가 이전 값인 1을 유지함을 확인할 수 있다.

RESET: j = 0, k = 1을 입력하고 clk 입력을 하여 q가 0, nq가 1을 출력하는 것을 확인할 수 있다.

TOGGLE: j = 1, k = 1일 때는 clk가 입력될 때마다 q가 0→1, 1→0으로 반전되는 것을 확인할 수 있다.

5. D Flip-Flop with Asynchronous Set and Clear

a. VHDL based implementation

assignment_3_5.vhd

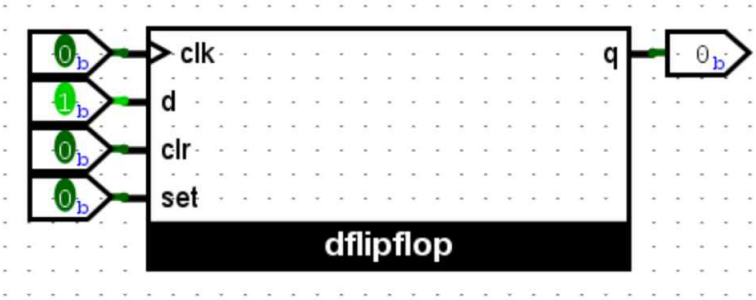
```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.all;
USE ieee.numeric_std.all;

ENTITY dflipflop IS
  PORT (
    clk : IN std_logic;  -- Clock signal
    d   : IN std_logic;  -- Data input
    clr : IN std_logic;  -- Asynchronous clear signal
    set : IN std_logic;  -- Asynchronous set signal
    q   : OUT std_logic  -- Data output
  );
END dflipflop;

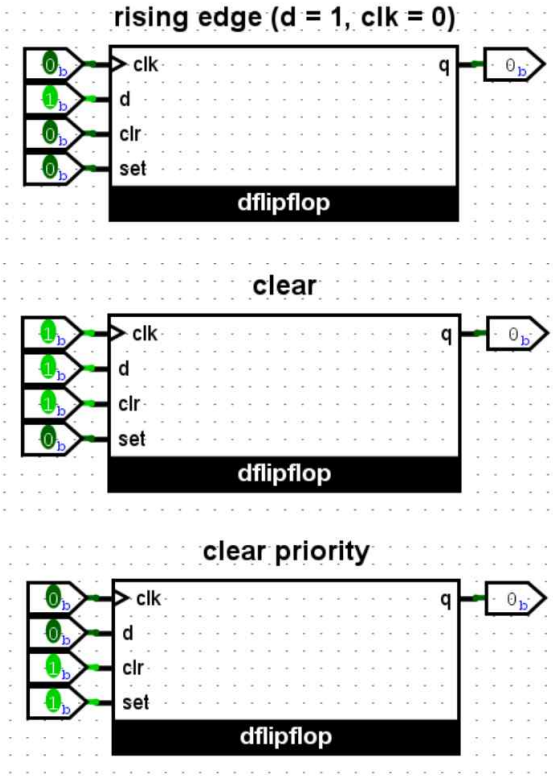
ARCHITECTURE bhv OF dflipflop IS
BEGIN
  process(clk, clr, set) -- Include clr and set for asynchronous handling
  BEGIN
    -- Asynchronous clear has the highest priority
    IF clr = '1' THEN
      q <= '0';
    -- Asynchronous set has the second-highest priority
    ELSIF set = '1' THEN
      q <= '1';
    -- Synchronous clock-based behavior
    ELSIF (clk'event AND clk = '1') THEN
      q <= d;
    END IF;
  END process;
END bhv;
```

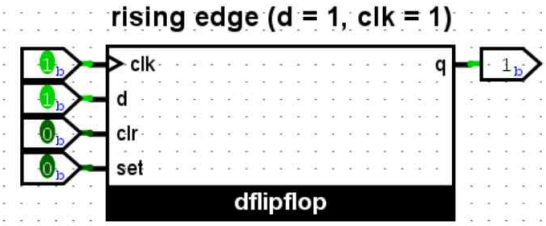
process의 감지 목록에 clr, set을 넣어서 clk와 관계없이 clr, set이 바뀌는 즉시 process가 실행되도록 했다. if문에서 clr를 가장 먼저 검사하여 clr가 set보다 우선순위가 높도록 했고, clr와 set이 모두 0일 때만 clk의 상승 에지에서 q <= d를 수행하는 일반 D Flip-Flop으로 동작한다.

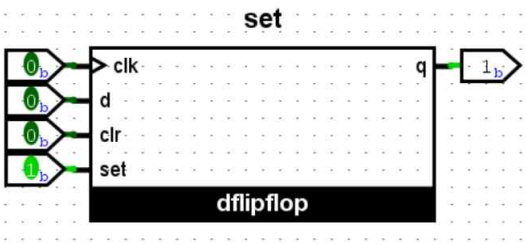
b. circuit screenshot



c. simulation screenshots







rising edge: clr, set이 0일 때의 일반 D 플립플롭 회로이다. d에 1을 넣고 clk를 0에서 1로 올리면 q에 1이 출력되는 것을 확인할 수 있다.

clear: q가 1인 상태에서 clk는 1로 그대로 두고 clr만 1로 바꾸었는데, 상승 에지 없이도 q가 즉시 0이 되는 것을 확인할 수 있다.

set: clk가 0인 상태에서 set을 1로 바꾸자 q가 즉시 1이 되는 것을 확인할 수 있다.

clear priority: clr와 set에 모두 1을 입력하면 clr가 우선하여 q가 0이 되는 것을 확인할 수 있다.